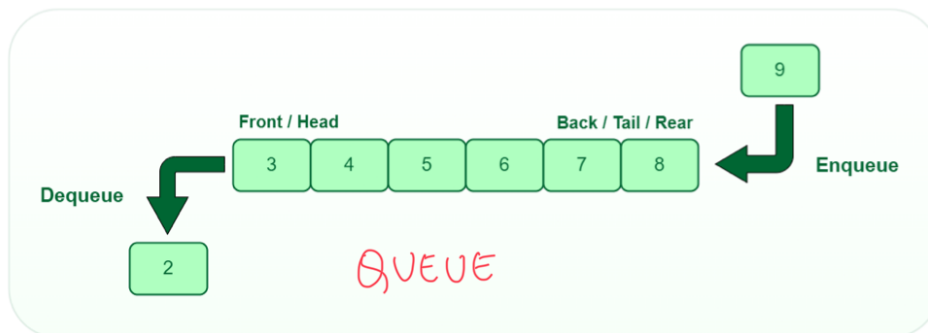
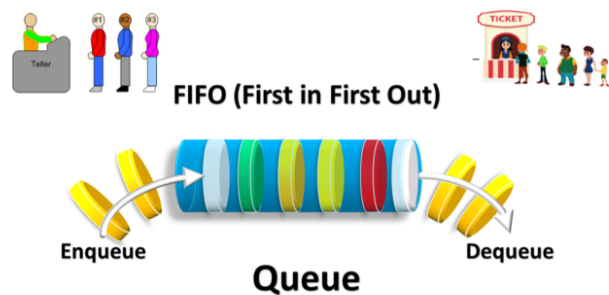
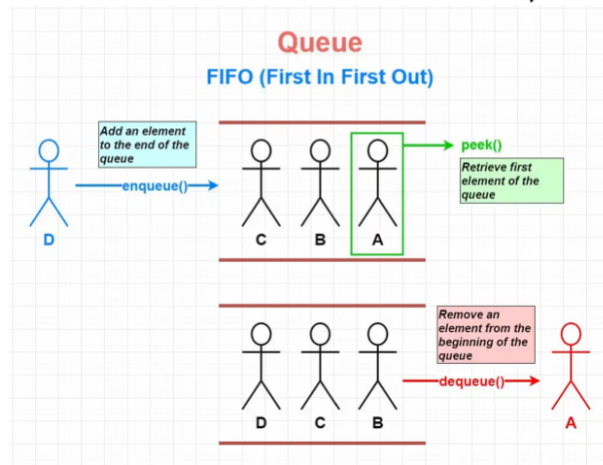


QUEUE

DAY 51
30/08/2025



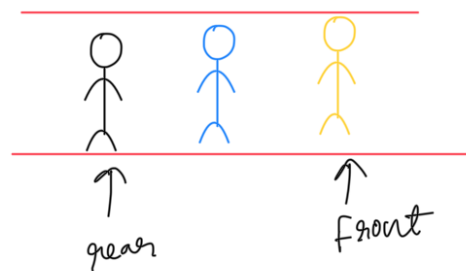
Similar to stack Queue is a linear Data structure that follows a Particular order in which the Operations are Performed for storing Data.

The Order is First in first Out (FIFO). One can Imagine a queue as a line of people waiting to receive something in Sequential order which starts from the Beginning of the line

- It is an ordered list in which insertions are done at one End which is known as the rear

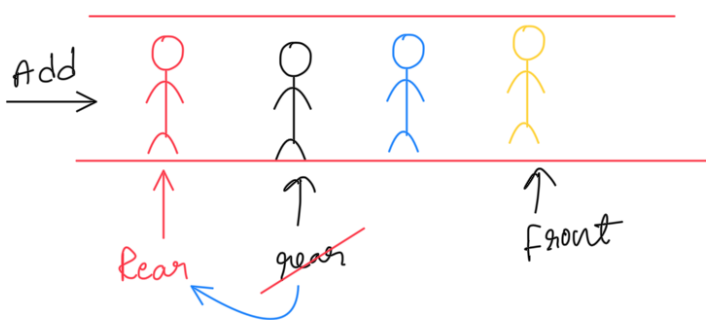
- and deletions are done from the other End known as the front.
- A good Example of a queue is any queue of consumers for a resource where the consumer that came first is served first
- The difference between stack and queue is in removing an element. In stack we remove the item that is most recently added, while in a queue we remove the item that is least recently added.

Queue Adding

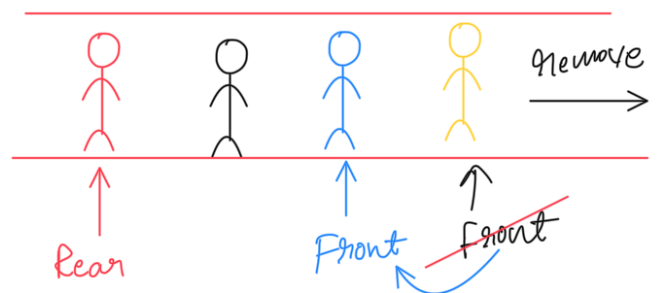


Array

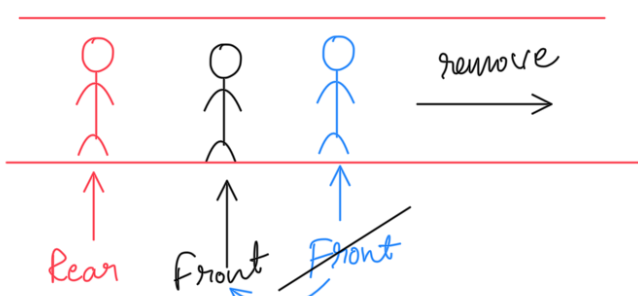
Add/Enqueue



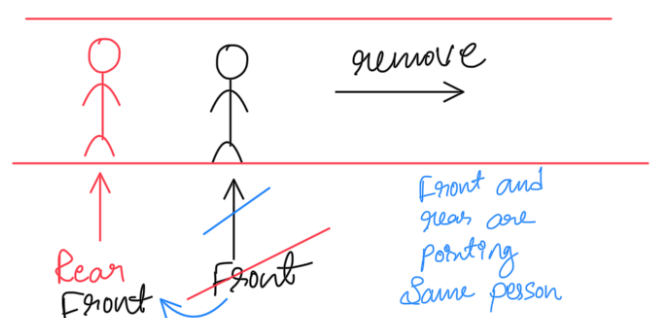
Remove / Dequeue



Remove / Dequeue



Remove / Dequeue



If Queue is Empty

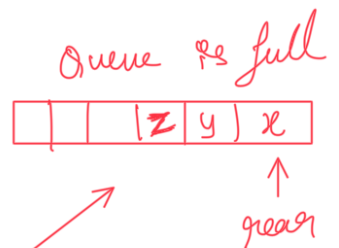
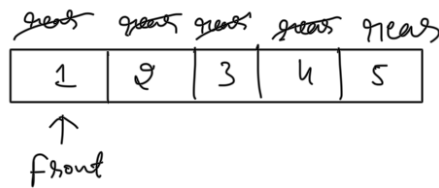
Front = Rear = -1

If the Queue is Empty the front & rear doesn't point to any valid index

Queue is full condition

Example

Capacity = 5



We just keep moving rear forward

if (rear == -1)

NOTE : Even if there are free slots in the front (after dequeues), we can't reuse them → WASTE of Space

That's why Circular Queue is preferred.

Queue Terminologies

→ Rear → Adding → Insertion → Enqueue
In Queue rear is for adding elements

→ Front → Removing → Deletion → Dequeue
In Queue front is for deleting elements

→ Capacity → max size
Capacity is the max size of the queue

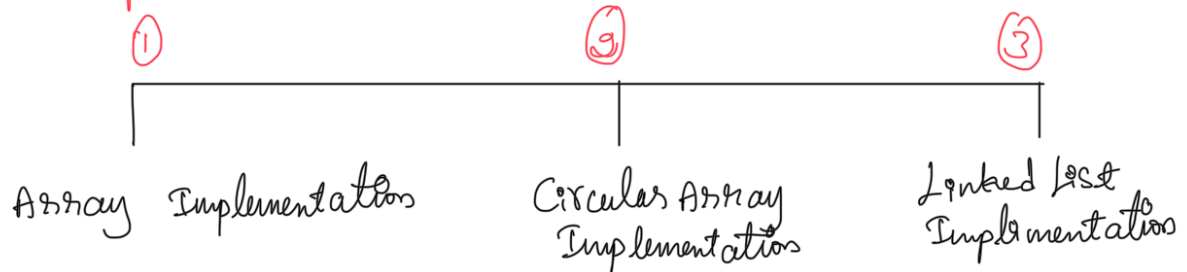
if front == -1 or
front > rear
Queue is Empty

→ Front = -1 or front > rear "Queue is Empty"
if front and rear are not pointing to any valid index then Queue is Empty

→ $\text{Rear} == \text{Size} - 1 \rightarrow$ Queue is full
if Rear is pointing to the last index
of the Array then the Queue is
full

→ $\text{Peek}() \rightarrow$ Top Element (Element At Front)
The $\text{peek}()$ function returns the Element At
Front of the Queue if Queue is non
Empty.

Queue Implementation



Queue can be Implemented in 3 different methods

- ① using Array.
- ② Using Array + Modulo wrapping around Array.
- ③ Linked List Implementation.

Queue Follows FIFO (First In First out)

The Element which is Added First will Be
Removed First.

All the operations of Queue
must be performed in $O(1)$
Constant time

Operation	Meaning	Complexity
Enqueue(x)	Insert element at rear	$O(1)$
Dequeue()	Remove element from front	$O(1)$
Peek() / Front()	Get front element without removing	$O(1)$
isEmpty()	Check if queue has no elements	$O(1)$
isFull() (in array impl)	Check if queue is full	$O(1)$

Tomorrow I will Implement the Queue using 3 methods.

Also let us understand the drawback of linear Queue Implementation and How the modular wrapping Helps to Overcome the issue.